

Gate 1 test results

A controlled two-arm test of Gate 1 inside Agent Laboratory. One YAML file, one environment switch, twenty executions, two complete papers. Every figure below is re-derived from retained artifacts at build time.

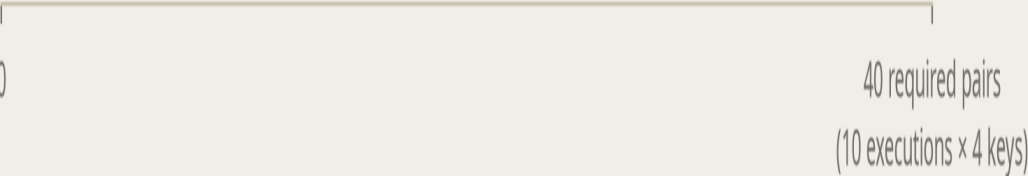
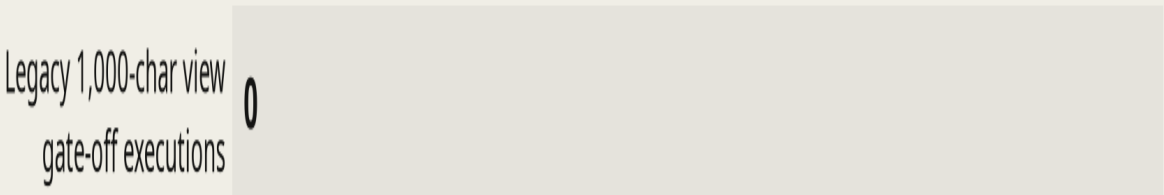
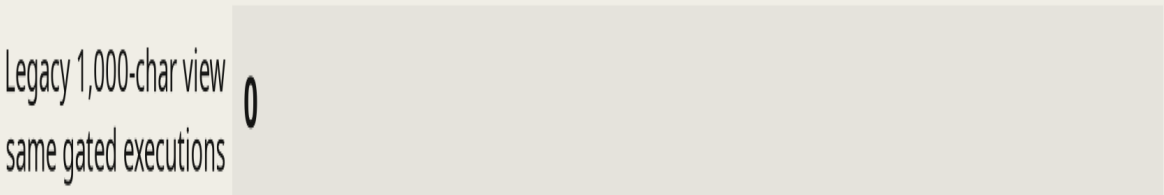
config a0409cf020f3... · 20 executions · 4 required keys per execution

01 **Headline measurements**

Four numbers, each with the control it is measured against. The first is the causal result: the same ten gated executions, read through two different channels.

40/40 required values delivered to the writer same executions, legacy channel: 0/40	96.6% gated paper claims traceable to the run gate-off paper: 0% of 11 claims	9/18 rejected turns the legacy rule could not see 36 executions across 15 adversarial scenario runs	3.73 / 3.77 reviewer means, gated / gate-off both papers recommended for rejection — no quality gain measured
--	---	--	---

Required key/value pairs delivered to the writing phase



Same artifacts, two channels. The gate-off arm is shown as the independent confirmation.

QUANTITY	GATE 1 ON	GATE 1 OFF	DIFFERENCE
Executions accepted	10 / 10	9 / 10	gate-off candidate 5 had a syntax error
Required pairs recorded to disk	40 / 40	36 / 40	+4
Required pairs delivered to the writer	40 / 40	0 / 40	+100 points

Same gated artifacts, legacy channel	0 / 40	—	the channel, not the arm, is the cause
Required values reported in the paper	4 / 4	0 / 4	+100 points
Paper claims traceable to the run	28 / 29 (96.6%)	0 / 11 (0%)	+96.6 points
Mean of three reviewer scores	3.735	3.765	−0.030 — both rejected

Recording is not delivery. Both arms called record_result and both wrote a registry to disk; the measured gain is entirely in whether anything downstream could read it.

02 What was tested

One Agent Laboratory workflow per arm, launched from the same YAML file. The environment switch is the entire treatment: same task, same models, same prompts, same result contract.

PARAMETER	GATE 1 ON	GATE 1 OFF	CONTROL
Shared configuration SHA-256	a0409cf020f37a9c...	a0409cf020f37a9c...	identical
Treatment	GATES_GATE1=on	GATES_GATE1=off	the only difference
All agent roles	deepseek-v4-flash	deepseek-v4-flash	same
Executions completed	10	10	same
Required result keys	4	4	same
Wall-clock	61 min	146 min	gate-off arm ran unparallelised on a retry
Papers produced	1	1	both complete, 7–8 sections

Gate 1 as integrated: 22 declared checks in 7 families (13 blocking, 6 warning, 3 informational), reached through a 450-line host adapter. Both arms are single workflows; the ten executions inside each are dependent optimisation steps, not ten independent research tasks.

03 Why the values did not arrive

The gate-off arm recorded every required value and lost all of them. The cause is structural rather than

numerical: upstream has exactly one route from a run to the writing agent, and it is a head slice of stdout.

ROUTE	INTEGRATION POINT	GATE 1 ON	GATE 1 OFF
typed registry (`results.json`)	gates/adapters/ agentlab.py:348	whole, unbudgeted, ahead of stdout	no reader exists
stdout head slice	gates/adapters/ agentlab.py:292	up to 40,000 chars, after the registry	first 1,000 chars, only route
solver hand-off	ai_lab_repo.py:390	`final.evidence_bundle`	`final.evidence_bundle`
writer prompt	papersolver.py:568	registry + stdout	1,000-char slice

The registry route does not exist with the gate off, and never did upstream — Agent Laboratory has no result registry, so there is no code path that could read one. The bypass reproduces that faithfully rather than simulating it.

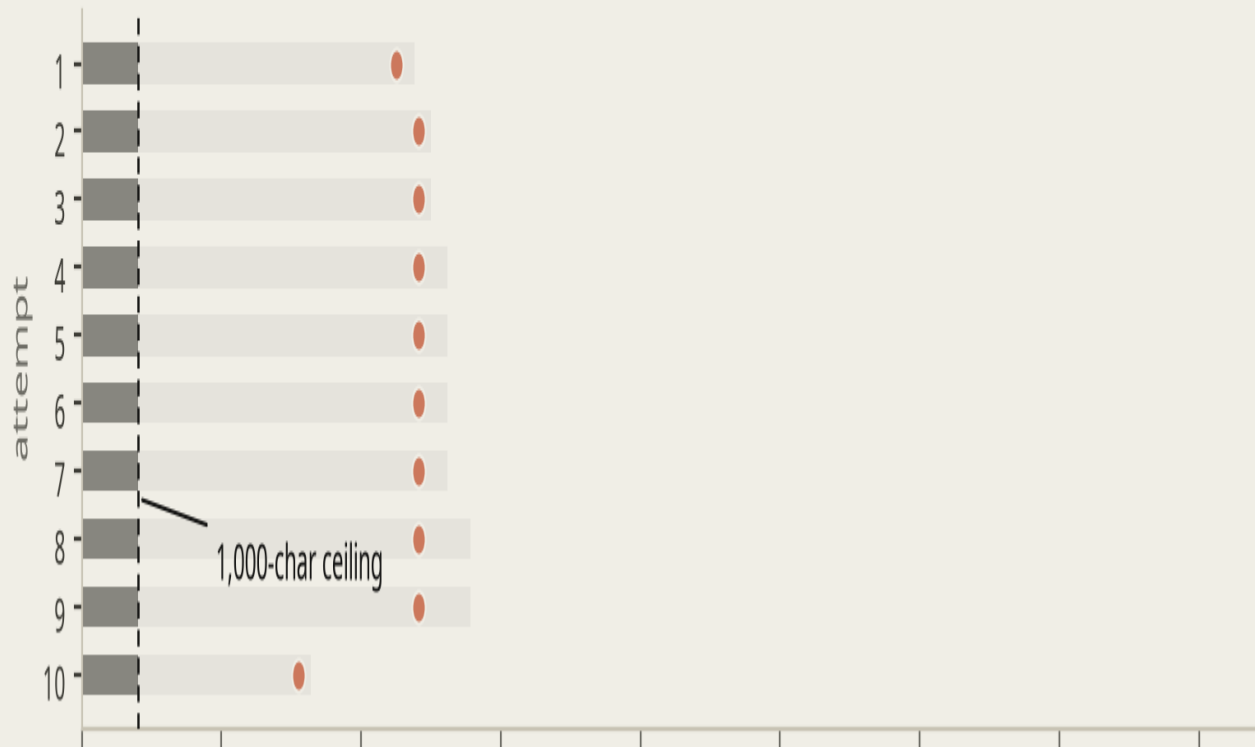
One execution, end to end — the gate-off arm's final run, ungated_10	
QUANTITY	VALUE
Values recorded to results.json	10 keys, 4 of them required (5,142 bytes on disk)
stdout captured	19,945 characters
stdout delivered to the writing phase	1,000 characters
Discarded	94.99%
First required value appears at character	14,807
Delivered slice ends at character	1,000 — mid-token, [arm=25][st

The block that was discarded is the one the engineer printed on purpose:

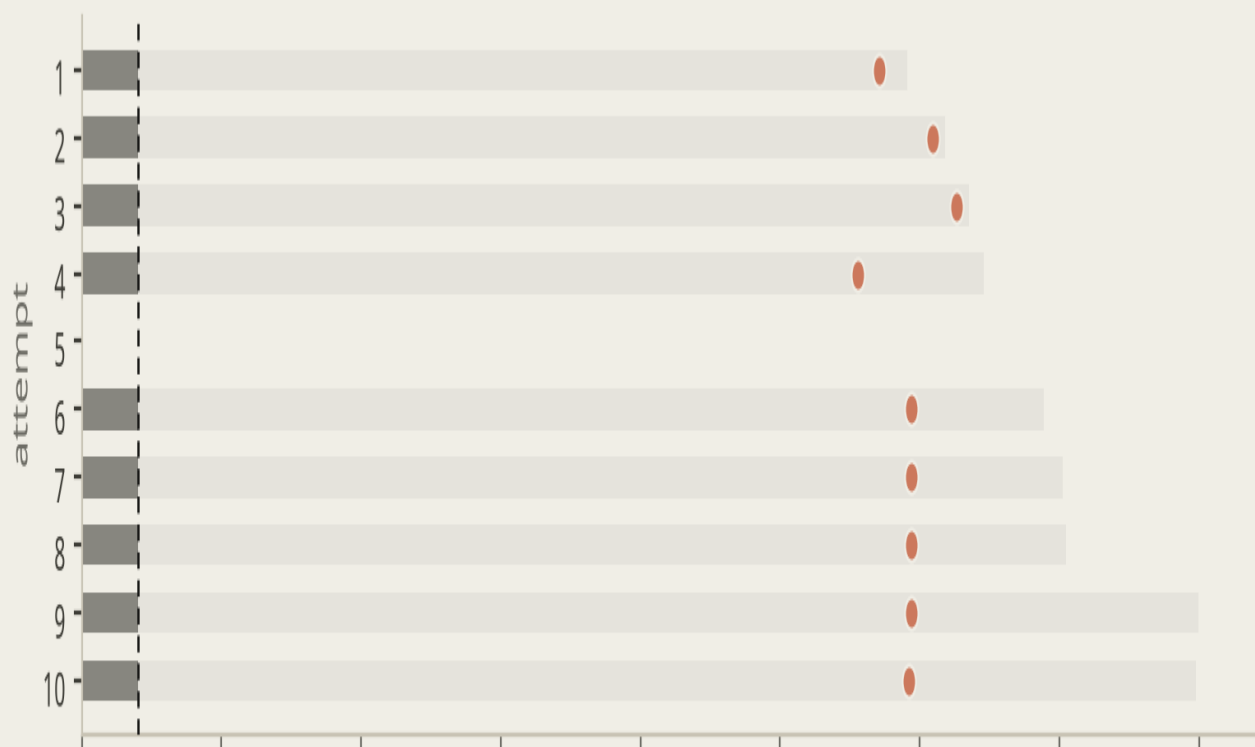
MEASURED RESULTS (from the controlled run): eff.acc_at_25 = 0.775000 eff.acc_at_100 = 0.810000 eff.efficiency_ratio = 0.956790 eff.train_s = 0.107033 seconds ----- Interpretation: 25% of the data retains at least 95% of the full-data

■ delivered under the legacy rule ■ captured but discarded ● first required value appears here

Gate 1 on — 10 executions



Gate 1 off — 10 executions



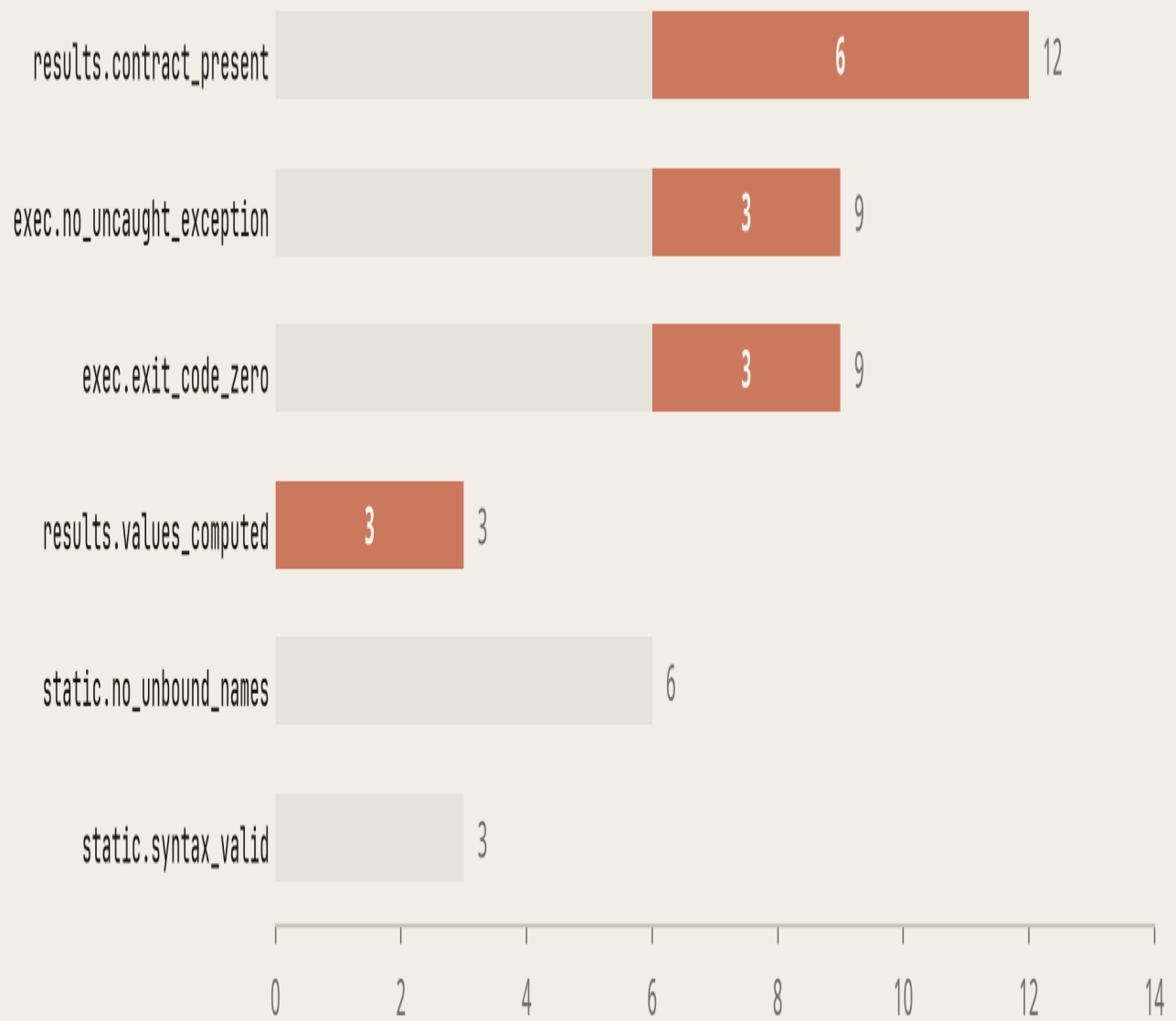
	GATE 1 ON	GATE 1 OFF
stdout captured across 10 executions	62,617 chars	154,596 chars
Executions whose required values sit past character 1,000	10 / 10	9 / 10
Required pairs a legacy channel would have delivered	0 / 40	0 / 40
Required pairs actually delivered	40 / 40	0 / 40

Gate 1 does not win this by printing less. Its own executions are equally invisible to the legacy rule — 10 of 10 sit past the boundary. It wins by not using stdout as the channel at all: the typed registry is handed over whole, ahead of any character budget.

04 Which checks did the work

No blocking check failed anywhere in the gated campaign: 10 of 10 executions passed all 13. An all-pass run cannot attribute anything — a check that never fails says nothing about what it catches — so attribution comes from the adversarial scenario corpus, run three times: 15 scenario runs, 36 executions, 24 failed executions across 18 rejected engineer turns, and 0 expectation mismatches.

Checks that rejected an execution, and what the legacy rule saw



blocking verdicts issued across 36 adversarial executions

grey legacy rule also caught it coral legacy rule was blind

Blocking verdicts by check. The coral portion is what the upstream substring rule could not have seen.

CHECK	FAMILY	TIER	FIRED	LEGACY BLIND	BLIND RATE	SCENARIOS
results.contract_present	results	FAIL	12	6	50.0%	archived-run, inner-repair,

						namespace-leak
exec.exit_code_zero	exec	FAIL	9	3	33.3%	archived-run, inner-repair, namespace-leak
exec.no_uncaught_exception	exec	FAIL	9	3	33.3%	archived-run, inner-repair, namespace-leak
results.values_computed	results	FAIL	3	3	100.0%	recovers
static.no_unbound_names	static	FAIL	6	0	0.0%	archived-run, recovers
static.syntax_valid	static	FAIL	3	0	0.0%	inner-repair

At the feedback-loop level, 9 of 18 rejected engineer turns (50%) were invisible to upstream's rule. At the check-execution level, that is 9 of 24 failed executions (38%). Two checks account for the reach that upstream structurally cannot have: `results.contract_present` and `results.values_computed` test the result contract, which upstream has no concept of. `static.no_unbound_names` and `static.syntax_valid` fired six and three times and were never blind — they add speed and a repair message, not reach.

The full declared surface follows, including the 16 checks that never issued a verdict in either artifact set. An unfired check is an untested one, so they are listed at their observed rate rather than dropped.

CHECK	FAMILY	TIER	WHAT IT ESTABLISHES	GATED CAMPAIGN	CORPUS
results.contract_present	results	FAIL	at least one typed result exists	10/10	12
exec.exit_code_zero	exec	FAIL	the child exited zero	10/10	9
exec.no_uncaught_exception	exec	FAIL	no exception escaped execution	10/10	9
static.no_unbound_names	static	FAIL	every referenced name resolves	10/10	6
results.values_computed	results	FAIL	the call-site argument is not a literal	10/10	3
static.syntax_valid	static	FAIL	source compiles before	10/10	3

execution is paid for				
env.clean_namespace	env	FAIL	execution began with harness 10/10 globals only	—
env.code_identity	env	FAIL	executed source hash 10/10 matches the recorded hash	—
exec.completed_within_budget	exec	FAIL	the process group finished 10/10 before timeout	—
results.contract_not_shadowed	results	FAIL	the injected recording API 10/10 was not redefined	—
results.expected_keys_present	results	FAIL	every declared key was 10/10 recorded	—
results.values_finite	results	FAIL	no NaN or infinity was 10/10 recorded	—
static.no_banned_calls	static	FAIL	no exit()/sys.exit() exit-code 10/10 forgery	—
env.seed_recorded	env	WARN	seed metadata is present 10/10	—
exec.no_swallowed_traceback	exec	WARN	no traceback was printed and 10/10 ignored	—
logs.model_error_signals	logs	WARN	model scan found residual not emitted suspicious lines	—
logs.no_error_signals	logs	WARN	deterministic scan found no 10/10 error signal	—
results.non_degenerate	results	WARN	no exact 0, 1, or chance-level 10/10 value	—
results.single_observation	results	WARN	a key was not silently 1/10 overwritten	—
env.provenance	env	INFO	runtime, platform, and library 10/10	—

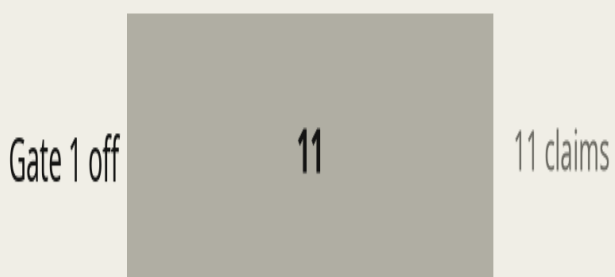
versions recorded					
output.untruncated	output	INFO	full stdout/stderr size and path recorded	10/10	—
report.fixes_grounded	report	INFO	ungrounded model-written fixes were discarded	not emitted	—

`results.single_observation` warned on 9 of 10 gated executions: Agent Laboratory prepends data-preparation code to the experiment body, so timing keys were recorded twice. The warning is correct and the underlying phase-scoping issue is open. WARN and INFO tiers can never change a verdict — a property enforced by an AST test that fails if Severity.FAIL appears anywhere in the model-facing layer.

05 What backs the numbers in each paper

Every numeric claim in each paper's findings sections was extracted and scored against that run's registry by the same procedure, with the same tolerance and the same claim filter.

Numeric claims in the findings sections, by what backs them



■ recorded in the registry
 ■ checkably derived from it
 ■ printed only — no registry
 ■ no origin found

The gate-off paper is scored on the most favourable basis available to it — its full stdout, including the 95% it never saw.

MEASURE	GATE 1 ON	GATE 1 OFF
Numeric claims extracted	29	11
Matched to a recorded result	19	0
Checkably derived from one	9	0
Printed only, bound to nothing	0	11

No origin found	1	0
Traceable to the run	96.6%	0.0%
Required values reported numerically	4 / 4	0 / 4
record_result calls in the accepted code	4	10

The gate-off rate is 0% on both the delivered basis and the full-stdout basis, because the arm has no registry route at all — there is nothing for a claim to be traceable *to*. The single unsourced gated claim is an external literature comparator, not a result.

06 Honesty came from the prompt, not from the gate

The gate-off paper did not fabricate the missing values. That is worth separating from the gate, because the instruction that produced it is part of the control and was byte-identical in both arms.

“Report a numeric value only when that value was supplied by the experiment evidence available to the writing phase. If a required value is absent from that evidence, write “not available from the execution evidence” in its table cell; do not estimate, reconstruct, or copy it from the plan.”

MEASURE	GATE 1 ON	GATE 1 OFF	ARCHIVED RUN
Shared report instruction present	yes	yes no — original prompts	
Required values fabricated	0	0	—
Numeric claims with no origin	1 / 29	0 / 11	65 / 65
Claims matched to a registry	19	0	0

The archived run, executed under Agent Laboratory's original prompts, invented all 65 of its figures. With the shared instruction in place, the gate-off arm instead declared the absence. The two arms therefore differ in **delivery**, not in honesty.

The instruction was not fully obeyed either.		
REQUIREMENT	GATE 1 ON	GATE 1 OFF
Four-row table with the exact key labels	no	no
Mandated phrase “not available from the execution evidence”	n/a — values present	0 occurrences

Tabular environments in the paper	1	2
-----------------------------------	---	---

The gate-off paper stated the absence in prose instead of the required table. Directionally honest, structurally non-compliant — which is why report-contract enforcement is listed as Gate 3 work rather than claimed as solved.

07 What did not improve

Three outcomes were measured and did not move. Reporting them is part of the result.

OUTCOME	GATE 1 ON	GATE 1 OFF	READING
Mean of three reviewer scores	3.735	3.765	−0.030; within noise of one workflow each
Reviewer decisions	3 × Reject	3 × Reject	no acceptance difference
Paper-writer reward	4.114	3.886	+0.227; one stochastic sample each
Numerical accuracy of visible values	exact	exact	deterministic task — nothing to improve
Scientific validity of the method	not checked	not checked	the gated paper fits an exponent to two points and passes
Task compliance	not checked	not checked	gated code imported a fixture despite a NumPy-only instruction

Gate 1 is an execution-validity and evidence-delivery gate. It establishes that a number came from a run that happened; it does not establish that the number means anything.

08 Implementation inventory

COMPONENT	FILES	LINES	NOTE
gates/ — the portable layer	19	4,630	no host imports outside adapters/
rig/ — measurement harness	11	2,878	scenarios, ablations, paper audit
tests/	12	3,018	211 test functions, 286 cases collected

Host adapter	1	450	the entire Agent Laboratory coupling
GATE 1 SURFACE	COUNT	DETAIL	
Declared checks	22	7 families: static, exec, env, results, logs, output	
Blocking (FAIL)	13	deterministic; decide the verdict alone	
Warning (WARN)	6	reported and passed downstream; cannot block	
Informational (INFO)	3	provenance and output accounting	
Model-assisted	1	log scan — WARN-only by construction	
Regression suites	286 + 76	gates + Agent Laboratory, as recorded at campaign close	

The one model-assisted component was benchmarked separately against a hand-labelled 68-line corpus (34 signal, 34 clean):

LOG SCANNER	PRECISION	RECALL	F1	MISSED SIGNALS
deterministic patterns	1.000	0.529	0.692	16
+ model scan (3-shot)	1.000	0.882	0.938	4

Recall +35.3 points, missed signals down 75%, zero measured false positives in either configuration. The finding remains WARN-only: the model can widen diagnostic coverage but cannot admit or reject an execution.

09 Benchmark context

MLR-Bench (arXiv 2505.19955) is the closest published measurement of the failure this layer targets. Its tasks and rubric are not this harness, so the figures below are context and not a leaderboard position.

MLR-BENCH FINDING	VALUE	GATE 1 COVERAGE
Coding-agent results fabricated or invalidated	80%	partial — execution and provenance only
Claude Code tasks using placeholder results	8/10	strong — exit, exception, timeout, code identity

Claude Code soundness / completeness / overall	4.75 / 6.00 / 4.95	below the 6.0 acceptance threshold
Result never entered an evidence channel	—	strong — 40/40 delivered vs 0/40 legacy
Placeholder value assigned to a variable	—	weak — indirect constants pass values_computed
Method is scientifically invalid	—	out of scope — Gate 2/3

No MLR-Bench task was run. Comparing this campaign's 0/10 blocking failures against MLR-Bench's 8/10 placeholder rate as though Gate 1 reduced the same benchmark by 80 points would be wrong: different task, different agent, explicit result-contract instructions in both arms here.

10 Limits on these results

LIMIT	STATEMENT
Sample size	One workflow per arm. The ten executions inside each are dependent optimisation steps, not ten independent tasks. No confidence interval is computed for the paper-level results.
values_computed is a literal check	record_result("x", 0.816) fails; acc = 0.816; record_result("x", acc) passes. It is not proof of measurement.
Expected keys are presence-only	Extra results do not fail. Two shadow-audited sources carried six undeclared metrics.
Phase boundaries blend	Data-preparation code is prepended to the experiment body, so an earlier phase's metrics can satisfy a later phase's contract. This is what results.single_observation warned about 9 times.
Task compliance is not checked	The gated final code imported a Hugging Face fixture against a NumPy-only instruction, executed cleanly, and passed.
The paper is not gated	Two-point exponent fits, single-seed generalisation claims, and report-contract violations all survive Gate 1 untouched.
Process isolation, not a sandbox	Experiment code runs as the same OS user. Credential-shaped variables are stripped and the parent is marked non-dumpable, but production use needs a container or unprivileged account.

Cost is unmeasured

Agent Laboratory printed \$0.0 as a placeholder; provider billing was not captured.

Defensible scope. Gate 1 is complete for deterministic execution validity, typed result capture, and causal provenance through the value layer. It is not a standalone anti-fabrication system, and these results do not support claiming that it eliminates fabricated science end to end.